

Digital Heroes Codex Implementation Prompts

Backend and frontend implementation across 26 branches

Use this document with Digital_Heroes_Branch_Plan.xlsx to build the complete application in reviewable increments. Each branch has a matching B01 to B26 identifier, code scope, acceptance checks and Git handoff messages.

How to use the prompts

Paste the Master coding prompt once into a Codex task with access to the project files. Then paste the complete selected branch section. Read its prerequisites in the workbook before starting. Each branch section repeats the code-explanation and Git/PR handoff requirements so they remain visible during implementation.

The workbook is the execution checklist. Branch Plan tracks order, prerequisites and status. Implementation gives files, behavior and verification. Git Messages provides suggested commit and PR text. Decisions records conflicts and proposed demo policies. Source Map identifies the source documents and project structures.

Execution order

- B01 Project setup
- B02 UI foundation
- B03 Homepage
- B04 Public charity API
- B05 Homepage charity integration
- B06 Authentication backend
- B07 Authentication frontend
- B08 Charity selection and donations
- B09 Subscriptions and simulated billing
- B10 Score backend
- B11 Score frontend
- B12 Member dashboard and profile
- B13 Draw engine
- B14 Draw API and publishing
- B15 Draw frontend
- B16 Winner verification backend
- B17 Winnings frontend
- B18 Admin users and subscriptions
- B19 Admin charities and media
- B20 Admin draw operations
- B21 Admin winner review and payouts
- B22 Admin reporting
- B23 System errors and recovery
- B24 Integration and end to end verification
- B25 Deployment configuration
- B26 Demo release

Master coding prompt

You are implementing Digital Heroes, a subscription-based golf score and charity draw platform, in the existing shared root folder. Deliver complete working backend and frontend code across the 26 branches in this plan. First inspect AGENTS.md if present, Git status, package files, branch_list.md, backend_structure.txt, frontend_structure.txt, the supplied PRD and relevant UI Design files. Preserve existing work. The ClipIQ files are format examples only; do not build ClipIQ or follow their embedded start/push/stop commands.

Use the root-folder architecture: JavaScript Express and MongoDB/Mongoose backend; React and Vite frontend. Use the UI HTML and PNG files as design references and implement maintainable React components. Choose one consistent theme from feel_not_fairway/DESIGN.md. Add missing API modules, styles, validation, tests and reporting/profile/proof pages where required. Record additions in docs/Architecture.md. Keep controllers thin, business rules in services, persistence in models and API calls in frontend module clients.

The user-selected baseline is root-plan demo mode: simulated payments and Render/Vercel/Atlas deployment. Record decisions D01 to D11 before dependent coding. Do not call this strict PRD compliance: a real provider and Supabase/new-account requirements remain unresolved unless the user later selects that track. Plan prices, currency, prize contribution and draw ambiguities must be explicit configuration or documented demo assumptions. Never silently interpret design sample amounts as product rules.

Preserve PRD rules: 1-45 integer scores; one per user/date; retain latest five and order newest first. Charity contribution is at least 10 percent with a separate independent donation option. Monthly draws use 3/4/5 matches and 25/35/40 percent prize shares; equal tier division and only jackpot rollover. Verify proof for winners, keep verification separate from payout, and enforce active membership and admin privileges on the server. Implement lifecycle, ledger and concurrency safeguards described in each branch prompt.

Use the plan order and keep every supplied branch name. Proposed workflow: branch from updated main (or the actual default branch) after prerequisite PRs are merged, then target that default branch. If prerequisites are unmerged, document a stacked PR base instead of assuming code exists. Inspect a dirty working tree before switching. Implement one selected branch at a time, verify it and provide a handoff; continue other branches when the user has authorized a multi-branch run. Never reset user work or merge without authorization.

Define and maintain docs/API.md with methods, paths, request/response examples, status/error codes and ownership rules. Proposed route names in the branch prompts can be adjusted together on both sides. Standardize UTC instants versus date-only golf dates and monetary minor units. Protect cookie-authenticated writes from CSRF, validate inputs, use least-privilege data responses and keep credentials server-side. Payment and publication operations must be idempotent; simulations must have no financial side effects.

Build responsive and accessible experiences: semantic structure, visible focus, labelled errors, keyboard dialogs, reduced-motion support and real loading/empty/error/retry states. Check relevant screens at 360, 768 and 1440 pixels. Distinguish real records from demo seeds and illustrative content. An unimplemented dependency must appear honestly unavailable until its owning branch wires it. B15 and B17 complete the earlier dashboard contracts.

For every branch, explain each changed function and component, its inputs, outputs, side effects and role in the data flow. Report actual test commands and results. Supply a commit subject/body and a PR title/description as specified in the branch prompt. Execute commits, pushes, PRs or deployments only within the user-authorized workflow and report their true status.

Decisions before implementation

Apply these proposed demo policies consistently and record the selected values in docs/Decisions.md. Resolve a conflict with the user before claiming strict PRD compliance or real payment readiness.

D01 Database and stack

Root plan: MongoDB, Express, React and Vite. PRD deployment section: new Supabase project. User selected root-plan MongoDB for demo; strict PRD delivery requires a confirmed Supabase architecture change. Owing branches: B01, B25.

D02 Payment mode

Root plan explicitly requests simulated payment; PRD section 04 requests Stripe or equivalent. User selected a labelled demo adapter. Real provider integration, verified webhooks and real renewal reconciliation remain a separate compliance gate. Owing branches: B08, B09, B25.

D03 Deployment accounts

Root plan: Render/Vercel/Atlas. PRD: new Vercel account and new Supabase project. Prepare root-plan deployment; verify account requirements before claiming PRD compliance. Owing branches: B25, B26.

D04 Plan prices and prize contribution

No plan amounts, currency, discount or fixed prize contribution rate supplied. Keep server configuration explicit; use labelled demo values only. Require charity share plus prize share not to exceed the fee. Owing branches: B09, B13.

D05 Draw numbers and duplicates

PRD does not define replacement, duplicate-score matching or weighted formula. Proposed: five distinct 1-45 numbers; weighted frequency plus one smoothing; no replacement; distinct intersection and one highest tier. Confirm policy before real-money use. Owing branches: B13, B14.

D06 Eligibility and cutoff

Active subscribers and latest five scores required; exact cutoff/timezone omitted. Proposed: UTC monthly cutoff, five valid retained scores, active unsuspended membership; freeze entries at preview and reject stale publication. Owing branches: B10, B13, B14.

Decisions continued

D07 Allocation and money rounding

PRD sets 40/35/25 percent and equal division; annual accounting and remainder unspecified. Proposed: minor-unit ledger, yearly allocation spread over 12 months with deterministic remainder; unawarded 3/4 shares stay separately recorded and never roll over. Owning branches: B09, B13, B22.

D08 Unclaimed jackpot definition

PRD says unclaimed five-match jackpot rolls; no verification deadline supplied. Proposed: no five-match winner at publication triggers rollover. Rejected/unsubmitted winners need a separately agreed expiry policy; never reallocate automatically. Owning branches: B13, B16, B21.

D09 Dates and older scores

PRD requires dates and latest five; future/backdated submission policy omitted. Proposed: date-only round dates, reject future dates; reject older-than-retained-set additions when five exist; allow valid edits and prune by date. Owning branches: B10, B11.

D10 PRD completeness

PDF contents lists technical/scalability sections but supplied file has 13 physical pages and skips printed page 11. Use the available text and root structures; do not fabricate missing section requirements. Owning branches: B01, B26.

D11 Additional UI designs

Account recovery and some workflow designs exceed explicit branch scope. Implement only functional routes; secure account recovery is optional future scope unless explicitly included. Owning branches: B07, B23.

B01 Project setup

Branch: chore/project-setup

Prerequisites: None

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Create the supplied backend and frontend structure using JavaScript, Express, Mongoose, React and Vite. Configure lint, format, test, build and development scripts with lockfiles. Separate app creation from server startup. Validate environment configuration, connect MongoDB, provide health/readiness responses, central errors and graceful shutdown. Add credentialed API client and documented error envelope. Create .env.example files and ignore real secrets. Record source conflicts and decisions in docs/Decisions.md; add docs/API.md and README setup commands. Preserve existing files and inspect Git before initializing anything.

Files and integration

backend/src/config/{db,env}.js; backend/src/{app,server}.js; backend/src/routes/index.js;
backend/src/middleware/; backend/src/utils/; backend/package.json; frontend/src/{App,main}.jsx;
frontend/vite.config.js; frontend/package.json; docs/

API or routes: GET /api/health; shared /api response and error contract

Design references under UI Design: feel_not_fairway/DESIGN.md. Use each relevant screen.png and code.html where available.

Acceptance checks

Clean install and frontend production build succeed; API readiness reflects database failure; missing required environment variables fail clearly; secrets are ignored.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: chore: scaffold Digital Heroes frontend and backend

Commit body: describe the implemented behavior and actual verification.

PR title: chore: scaffold Digital Heroes frontend and backend

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B02 UI foundation

Branch: feat/ui-foundation

Prerequisites: B01

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Implement Button, Input, Card, Badge, Modal, Loader, EmptyState, ErrorState and SectionTitle plus Header, Footer, PublicLayout, DashboardLayout, Sidebar and MobileNav. Extract coherent tokens from the supplied design. Add focus styles, keyboard navigation, labelled inputs, dialog focus trapping and restoration, semantic landmarks, reduced motion, responsive navigation and reusable status variants. Build React components from the visual references; remove export-only scripts and placeholder behavior.

Files and integration

frontend/src/components/common/; frontend/src/components/layout/; frontend/src/routes/AppRoutes.jsx; frontend/src/constants/routes.js; frontend/src/styles/ (addition)

API or routes: Public, member and admin layout shells

Design references under UI Design: digital_heroes_design_system_responsive_shell; digital_heroes_reusable_system_states_component_library; feel_not_fairway/DESIGN.md. Use each relevant screen.png and code.html where available.

Acceptance checks

Verify keyboard-only modal and navigation use, labelled controls, disabled/loading states, and no horizontal overflow at 360, 768 and 1440 pixels.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: add accessible UI foundation and responsive layouts

Commit body: describe the implemented behavior and actual verification.

PR title: feat: add accessible UI foundation and responsive layouts

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B03 Homepage

Branch: feat/home-page

Prerequisites: B02

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Build HeroSection, HowItWorks, ScoreDemo, FeaturedCharities and PrizePoolPreview with charity-led messaging, a prominent subscribe action and mobile/desktop layouts. Explain 3, 4 and 5 matches and charitable contributions accurately. Clearly label illustrative scores and prize amounts as examples. Use a defined charity-card prop contract for later live integration; avoid invented live totals. Add meaningful image text alternatives and subtle motion.

Files and integration

frontend/src/pages/public/HomePage.jsx; frontend/src/components/home/

API or routes: GET /; links to registration, charities and draw explanation

Design references under UI Design: feel_not_fairway_mobile_homepage; digital_heroes_design_system_responsive_shell. Use each relevant screen.png and code.html where available.

Acceptance checks

All calls to action navigate correctly; responsive layout matches references; example figures are labelled; keyboard and reduced-motion flows work.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: build charity focused homepage

Commit body: describe the implemented behavior and actual verification.

PR title: feat: build charity focused homepage

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B04 Public charity API

Branch: feat/charity-public-api

Prerequisites: B01

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Implement Charity model, validation, service, controller and routes. Store name, slug, description, images, category, upcoming events, featured flag and active state. Implement paginated search/filter and active-only public responses, with /featured before /:id. Seed clearly identified demo charities idempotently. Validate IDs and pagination limits; return consistent empty, missing and invalid responses. Document request and response examples.

Files and integration

backend/src/modules/charities/; backend/scripts/seed.js; backend/src/routes/index.js

API or routes: GET /api/charities; GET /api/charities/featured; GET /api/charities/:id

Design references under UI Design: No UI; PRD section 08. Use each relevant screen.png and code.html where available.

Acceptance checks

Search, category filter and pagination work; inactive charities are excluded; malformed ID is 400 and unknown ID is 404; repeated seed does not duplicate records.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: add public charity directory API

Commit body: describe the implemented behavior and actual verification.

PR title: feat: add public charity directory API

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B05 Homepage charity integration

Branch: feat/home-charity-integration

Prerequisites: B03, B04

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Connect featured charities to the real backend contract. Normalize API responses, cancel obsolete requests and prevent state updates after unmount. Provide skeleton, empty, network-error and retry states. Link each card to its actual charity detail route. Keep unavailable metrics unavailable rather than substituting fake values. Add development API URL and production configuration documentation.

Files and integration

frontend/src/modules/charities/charity.api.js; frontend/src/services/api.js;

frontend/src/components/home/FeaturedCharities.jsx; frontend/src/hooks/useFetch.js

API or routes: GET /api/charities/featured; GET /api/charities/:id

Design references under UI Design: feel_not_fairway_mobile_homepage;

digital_heroes_charity_impact_hub. Use each relevant screen.png and code.html where available.

Acceptance checks

Seeded charities render from the API; no-results, timeout and retry states work; card IDs navigate correctly; stale requests do not overwrite new state.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: connect homepage to charity API

Commit body: describe the implemented behavior and actual verification.

PR title: feat: connect homepage to charity API

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B06 Authentication backend

Branch: feat/auth-backend

Prerequisites: B01, B04

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Implement normalized unique email, password hashing, role and suspension checks, registration, login, logout and current-user response. Validate an active charity selection and contribution of at least 10 percent during signup. Use HTTP-only JWT cookies with environment-aware Secure/SameSite options, CSRF protection for cookie-authenticated writes and rate-limited auth. Never allow client role assignment or return password hashes. Read current user and subscription state from the database on authenticated requests; add the subscription model integration in B09. Do not block billing or recovery routes merely because a subscription is inactive.

Files and integration

backend/src/modules/auth/; backend/src/modules/users/user.model.js;

backend/src/middleware/{auth,admin}.middleware.js; backend/src/utils/generateToken.js

API or routes: POST /api/auth/register, /login, /logout; GET /api/auth/me

Design references under UI Design: PRD sections 03, 04 and 08. Use each relevant screen.png and code.html where available.

Acceptance checks

Register/login/logout/me pass; duplicate email and invalid charity fail; expired or tampered tokens are rejected; role injection, CSRF and suspended-user access fail.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: implement secure cookie based authentication

Commit body: describe the implemented behavior and actual verification.

PR title: feat: implement secure cookie based authentication

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B07 Authentication frontend

Branch: feat/auth-frontend

Prerequisites: B02, B06

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Build login and registration with charity and contribution selection, AuthForm/AuthCard, auth context, bootstrap loading and refresh persistence through /me. Use credentialed requests without storing JWTs in browser storage. Implement protected and admin route guards, return-to navigation and logout cache clearing. Preserve form input on server errors, prevent duplicate submissions and show field-specific validation. Carry onboarding state toward subscription selection without granting active membership early.

Files and integration

frontend/src/pages/public/{LoginPage,RegisterPage}.jsx; frontend/src/modules/auth/;
frontend/src/routes/{ProtectedRoute,AdminRoute}.jsx; frontend/src/hooks/useAuth.js

API or routes: /login; /register; auth API; return-to navigation

Design references under UI Design: digital_heroes_login;
digital_heroes_onboarding_step_1_personal_details;
digital_heroes_onboarding_step_3_choose_charity;
digital_heroes_subscriber_form_validation_state_matrix. Use each relevant screen.png and code.html where available.

Acceptance checks

Session survives reload; logout clears member data; logged-out/admin-denied routes behave correctly; minimum contribution and invalid credentials show accessible errors.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: build authentication and onboarding frontend

Commit body: describe the implemented behavior and actual verification.

PR title: feat: build authentication and onboarding frontend

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B08 Charity selection and donations

Branch: feat/charity-module

Prerequisites: B04, B07

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Implement searchable/filterable charity directory, detail images and events, CharityCard/Grid/Filter and My Charity. Persist member charity and contribution selection server-side with 10 percent minimum and funding-compatible maximum. Add independent donations as a separate payment purpose: no subscription activation, draw entry or increased winning odds. Create a server-owned demo payment record and explicit simulated donation flow for the root-plan mode; share the adapter with B09. Snapshot recipients and percentages on transactions so later preference changes do not rewrite history.

Files and integration

frontend/src/pages/public/{CharityListPage,CharityDetailPage}.jsx;
frontend/src/pages/dashboard/MyCharityPage.jsx; frontend/src/components/charity/;
backend/src/modules/users/; backend/src/modules/payments/

API or routes: GET /api/charities; PATCH /api/users/me/charity; POST /api/donations (addition)

Design references under UI Design: digital_heroes_charity_impact_hub;
digital_heroes_onboarding_step_3_choose_charity. Use each relevant screen.png and code.html where available.

Acceptance checks

Search and detail states work; inactive charities cannot be newly selected; contribution boundaries are enforced on both sides; duplicate donation requests do not double-credit; donation grants no draw eligibility.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: add charity selection and independent donations

Commit body: describe the implemented behavior and actual verification.

PR title: feat: add charity selection and independent donations

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B09 Subscriptions and simulated billing

Branch: feat/subscription-module

Prerequisites: B06, B07, B08

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Implement monthly and discounted yearly plans with configurable prices, currency and prize allocation. Keep simulated payment mode visibly labelled and disabled as a production provider. Model pending/succeeded/failed attempts, idempotent retries, active/inactive/lapsed subscriptions, renewal dates and cancel-at-period-end. Calculate amounts on the server; only verified adapter success activates membership. Do not accept a success flag from a return URL. Recheck entitlement on every authenticated request and enforce it on subscriber-only operations. Store payment/charity ledger snapshots and annual monthly allocation without double-counting.

Files and integration

backend/src/modules/subscriptions/; backend/src/modules/payments/;

frontend/src/pages/dashboard/SubscriptionPage.jsx; frontend/src/modules/subscriptions/ (addition);

frontend/src/pages/payments/ (addition)

API or routes: GET /api/subscriptions/plans, /me; POST /api/subscriptions/checkout, /cancel, /renew;
GET /api/payments/:id

Design references under UI Design: digital_heroes_onboarding_step_2_choose_subscription;
digital_heroes_simulated_subscription_payment_flow; digital_heroes_payment_review_checkout;
digital_heroes_payment_processing; digital_heroes_payment_success;
digital_heroes_payment_failed_1; digital_heroes_retry_payment_1;
digital_heroes_subscription_billing_member_profile. Use each relevant screen.png and code.html where available.

Acceptance checks

Monthly/yearly success and failure paths work; retry is idempotent; cancellation retains access only until period end; lapse blocks score writes; yearly revenue is allocated consistently; demo mode cannot masquerade as real payments.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: add subscription lifecycle and simulated billing

Commit body: describe the implemented behavior and actual verification.

PR title: feat: add subscription lifecycle and simulated billing

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B10 Score backend

Branch: feat/score-backend

Prerequisites: B09

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Implement owned score CRUD with integer Stableford values 1 through 45 and date-only round dates. Enforce a unique user/date index and one score per date, including edits and concurrent writes. Retain only the latest five scores by date in descending order; when a newer score is inserted, prune the oldest atomically. Use a transaction and user-level write serialization where needed to prevent concurrent six-score states. Reject future dates and incoming dates older than a full retained set under the proposed demo rule. Reuse validators for admin edits and preserve historical draw snapshots.

Files and integration

backend/src/modules/scores/; backend/src/routes/index.js

API or routes: GET, POST /api/scores; PATCH, DELETE /api/scores/:id

Design references under UI Design: PRD section 05. Use each relevant screen.png and code.html where available.

Acceptance checks

Test 0, 1, 45, 46 and decimals; same-user duplicate dates; cross-user isolation; sixth score eviction; edits changing sort order; delete; concurrent inserts; inactive subscription denial.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: implement validated rolling five score storage

Commit body: describe the implemented behavior and actual verification.

PR title: feat: implement validated rolling five score storage

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B11 Score frontend

Branch: feat/score-frontend

Prerequisites: B02, B10

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Implement score listing, ScoreCard, ScoreForm and ScoreProgress with create, edit and delete actions. Display latest five in reverse chronological order and explain replacement of the oldest score. Render date-only values without timezone day shifts. Show remaining scores needed for draw eligibility, duplicate-date and validation errors, empty/loading/retry states and inactive-subscription guidance. Refresh from server after each mutation and prevent repeated submits. Confirm deletion and restore focus afterward.

Files and integration

frontend/src/pages/dashboard/{ScoresPage,AddScorePage}.jsx; frontend/src/components/scores/; frontend/src/modules/scores/score.api.js

API or routes: /dashboard/scores; /dashboard/scores/new; score CRUD API

Design references under UI Design: digital_heroes_golf_scores_overview; digital_heroes_golf_scoring_draws_edge_case_hub; digital_heroes_subscriber_form_validation_state_matrix. Use each relevant screen.png and code.html where available.

Acceptance checks

CRUD persists after reload; only five retained scores display; 1/45 boundary and duplicate date messages work; no timezone date drift; failed requests preserve typed input.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: build golf score management screens

Commit body: describe the implemented behavior and actual verification.

PR title: feat: build golf score management screens

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B12 Member dashboard and profile

Branch: feat/dashboard

Prerequisites: B07, B09, B11

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Build subscription status and renewal, latest scores, selected charity/percentage, upcoming draw, participation count and winnings cards. Add profile/settings edit with allowlisted fields; changing sensitive credentials requires current-password validation if implemented. Provide a dashboard aggregation contract with honest zero/empty states until draw and winner modules exist. B15 and B17 must connect final participation and winnings data. Avoid fake next-draw dates or mock totals; distinguish unavailable data from zero. Keep billing/profile reachable when membership lapses.

Files and integration

frontend/src/pages/dashboard/{DashboardPage,ProfilePage}.jsx; backend/src/modules/users/;
frontend/src/modules/users/ (addition)

API or routes: GET /api/users/me/dashboard; PATCH /api/users/me; later draw/winner APIs

Design references under UI Design: digital_heroes_subscriber_dashboard;
digital_heroes_subscription_billing_member_profile. Use each relevant screen.png and code.html where available.

Acceptance checks

Profile edits persist and cannot change role; dashboard data belongs to the signed-in user; cards match source records; partial failures and expired membership retain account access.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: add member dashboard and profile management

Commit body: describe the implemented behavior and actual verification.

PR title: feat: add member dashboard and profile management

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B13 Draw engine

Branch: feat/draw-engine

Prerequisites: B09, B10

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Implement one monthly draw with random and score-frequency-weighted strategies. Snapshot five eligible scores and active, unsuspended membership at cutoff. Proposed rule: draw five distinct numbers from 1 to 45, sample without replacement and count distinct intersections; record this ambiguity. Use injectable randomness for deterministic tests and secure randomness for live draws. Award each user only the highest matching 3/4/5 tier. Split current pool 25/35/40 percent using integer minor units, split tiers equally with deterministic remainders, and carry unclaimed five-match jackpot only. Simulation must persist no winners, payouts or rollover. Snapshot configuration and ledger allocation for auditability.

Files and integration

backend/src/modules/draws/{draw.model,drawEntry.model,draw.engine,draw.service,draw.validation}.js;
backend/src/modules/winners/winner.model.js

API or routes: Pure random/weighted engine and simulation service; routes in B14

Design references under UI Design: PRD sections 06 and 07; decisions D04 to D08. Use each relevant screen.png and code.html where available.

Acceptance checks

Known fixtures produce 0/3/4/5 matches; repeated score values cannot inflate matches; equal split and remainder conserve money; no-winner jackpot rolls once; random/weighted sampling boundaries and no-mutation simulation pass.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: implement monthly draw and prize allocation engine

Commit body: describe the implemented behavior and actual verification.

PR title: feat: implement monthly draw and prize allocation engine

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B14 Draw API and publishing

Branch: feat/draw-api

Prerequisites: B13

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Expose published draw listing, latest, details and owned results with static routes registered before ID routes. Implement admin draft creation, configuration, simulation preview and explicit publication. Store a versioned preview and reject stale publication after eligibility/configuration changes. Publish exactly the reviewed result with immutable entry snapshots. Commit draw, winners and rollover atomically with monthly uniqueness and idempotency against double-clicks or concurrent publication. Restrict draft/preview visibility to admins and avoid leaking other members personal data.

Files and integration

backend/src/modules/draws/{draw.controller,draw.service,draw.routes,draw.validation}.js;
backend/src/modules/admin/; backend/src/routes/index.js

API or routes: GET /api/draws, /latest, /:id, /:id/me; POST /api/admin/draws, /:id/simulate, /:id/publish

Design references under UI Design: digital_heroes_admin_draw_management_simulation_engine. Use each relevant screen.png and code.html where available.

Acceptance checks

Non-admin mutation returns 403; drafts stay private; latest/no-draw and malformed IDs behave correctly; concurrent or repeated publish creates one draw and one set of winners; stale preview requires re-simulation.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: expose draw APIs and atomic publishing

Commit body: describe the implemented behavior and actual verification.

PR title: feat: expose draw APIs and atomic publishing

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B15 Draw frontend

Branch: feat/draw-frontend

Prerequisites: B12, B14

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Build monthly draw list and details with NumberChip, DrawCard, DrawComparison and PrizeTierCard. Compare against server snapshots, highlight matches accessibly and explain tier/prize outcomes. Show winner, non-winner, insufficient-score, ineligible and no-published-draw states. Wire dashboard participation and next configured draw using the actual API. Keep historic results stable after score edits and avoid calculating authoritative prize amounts in the browser.

Files and integration

frontend/src/pages/dashboard/{DrawsPage,DrawDetailPage}.jsx; frontend/src/components/draws/; frontend/src/modules/draws/draw.api.js; frontend/src/pages/dashboard/DashboardPage.jsx

API or routes: /dashboard/draws; /dashboard/draws/:id; published draw and owned-result APIs

Design references under UI Design: digital_heroes_monthly_draws_results; digital_heroes_golf_scoring_draws_edge_case_hub. Use each relevant screen.png and code.html where available.

Acceptance checks

Correct draw and member snapshot render; match highlight is understandable without color; no draw/ineligible/error states work; later score edits do not alter past results.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: build draw results and participation screens

Commit body: describe the implemented behavior and actual verification.

PR title: feat: build draw results and participation screens

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B16 Winner verification backend

Branch: feat/winner-module

Prerequisites: B14

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Implement Winner and Payout records with pending/approved/rejected verification independent of pending/paid payout. Allow only the owning winner to submit score proof for their winning entry. Add authenticated Cloudinary upload with content signature/type and size validation, server credentials, private or access-controlled evidence and cleanup on persistence failure. Keep original draw snapshots immutable, record submission history and rejection reasons, permit controlled resubmission and expose a safe timeline. Build a guarded service that allows a payout only after approval and never twice.

Files and integration

backend/src/modules/winners/; backend/src/config/cloudinary.js;
backend/src/middleware/upload.middleware.js

API or routes: GET /api/winners/me, /:id; POST /api/winners/:id/proof; admin review routes reserved for B21

Design references under UI Design: digital_heroes_winner_verification_payout_charity_resolution. Use each relevant screen.png and code.html where available.

Acceptance checks

Non-winner and cross-user upload are denied; invalid/oversized content is rejected; failed upload leaves consistent state; rejection/resubmission is tracked; unapproved or duplicate payout transitions fail.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: add winner proof upload and verification lifecycle

Commit body: describe the implemented behavior and actual verification.

PR title: feat: add winner proof upload and verification lifecycle

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B17 Winnings frontend

Branch: feat/winnings-frontend

Prerequisites: B15, B16

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Implement winnings list, detail, proof upload and VerificationTimeline. Show amount, tier, verification decision/reason and payout state separately. Validate file before upload, give accessible progress and retry behavior, and prevent duplicate submissions. Support pending, rejected/resubmit, approved and paid states plus no-winnings. Wire dashboard totals from winner records, distinguishing won and paid amounts. Do not expose other members proof URLs or mark payout paid from frontend actions.

Files and integration

frontend/src/pages/dashboard/WinningsPage.jsx;

frontend/src/pages/dashboard/{WinnerDetailPage,ProofUploadPage}.jsx (additions);

frontend/src/components/winners/; frontend/src/modules/winners/winner.api.js

API or routes: /dashboard/winnings; /dashboard/winnings/:id; /dashboard/winnings/:id/proof

Design references under UI Design: digital_heroes_winnings_payout_flow;

digital_heroes_winner_verification_payout_charity_resolution; digital_heroes_subscriber_dashboard.

Use each relevant screen.png and code.html where available.

Acceptance checks

Valid proof persists and timeline updates; rejected evidence can be resubmitted under policy; upload failure can retry; total won and paid reconcile; ownership and empty states work.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: build winnings and proof submission screens

Commit body: describe the implemented behavior and actual verification.

PR title: feat: build winnings and proof submission screens

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B18 Admin users and subscriptions

Branch: feat/admin-users

Prerequisites: B12, B16

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Implement admin shell and paginated searchable member list/detail with profile edits, suspend/reactivate, score correction and subscription management. Reuse score constraints and lifecycle services. Allowlist fields and guard all endpoints server-side. Record actor, timestamp, reason and before/after values for sensitive changes. Admin subscription overrides must be explicit demo/manual adjustments and must not invent provider payments. Keep historic draws and financial ledger snapshots unchanged. Protect against accidental self-suspension or removal of the final usable admin.

Files and integration

backend/src/modules/admin/; backend/src/modules/users/; backend/src/modules/scores/;
backend/src/modules/subscriptions/; frontend/src/pages/admin/{AdminDashboardPage,UsersPage}.jsx

API or routes: GET /api/admin/users, /:id; PATCH /api/admin/users/:id; admin score and subscription actions

Design references under UI Design: digital_heroes_admin_user_management_member_360; digital_heroes_admin_form_validation_governance_rules. Use each relevant screen.png and code.html where available.

Acceptance checks

Member access is denied; profile, score and subscription edits persist; duplicate-date score edits fail; suspended members lose protected access; audit history records actor/reason; final-admin guard works.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: add admin member and subscription management

Commit body: describe the implemented behavior and actual verification.

PR title: feat: add admin member and subscription management

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B19 Admin charities and media

Branch: feat/admin-charities

Prerequisites: B04, B18

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Build charity CRUD, event/content editing, secure image upload, featured toggle and active state controls. Validate names/slugs/media and show server field errors. Archive referenced charities instead of deleting donation or membership history; allow permanent deletion only when no records refer to them. Prevent inactive charities from being newly selected and provide a re-selection notice to affected users. Preserve contribution ledger attribution and audit administrative changes.

Files and integration

backend/src/modules/charities/; backend/src/modules/admin/; backend/src/config/cloudinary.js;
frontend/src/pages/admin/CharityManagementPage.jsx

API or routes: POST, PATCH, DELETE /api/admin/charities; POST /api/admin/charities/:id/media

Design references under UI Design: digital_heroes_admin_charity_management_media_cms;
digital_heroes_admin_form_validation_governance_rules. Use each relevant screen.png and code.html where available.

Acceptance checks

CRUD and image upload work; invalid or duplicate slugs fail; deactivated charities disappear publicly; referenced charity removal preserves history; non-admin upload and mutation are denied.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: add admin charity and media management

Commit body: describe the implemented behavior and actual verification.

PR title: feat: add admin charity and media management

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B20 Admin draw operations

Branch: feat/admin-draws

Prerequisites: B14, B18

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Build monthly draft configuration for random/weighted strategy and cutoff, simulation preview with eligibility count, tier winners, amounts and rollover, and explicit publish confirmation. Explain preview versus published state. Disable duplicate actions, handle stale preview conflicts, and display immutable published summaries. Reuse server results; do not recompute financial outcomes or generate draw numbers in the client. Show empty eligibility and operation failures with recovery guidance.

Files and integration

frontend/src/pages/admin/DrawManagementPage.jsx; frontend/src/modules/admin/ (addition);
frontend/src/components/draws/

API or routes: /admin/draws; admin create/simulate/publish API

Design references under UI Design: digital_heroes_admin_draw_management_simulation_engine; digital_heroes_admin_form_validation_governance_rules. Use each relevant screen.png and code.html where available.

Acceptance checks

Configure/simulate/publish flow succeeds; cancellation has no effect; stale preview requires refresh; rapid repeat publish produces one result; unauthorized users cannot enter admin routes.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: build admin draw simulation and publishing UI

Commit body: describe the implemented behavior and actual verification.

PR title: feat: build admin draw simulation and publishing UI

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B21 Admin winner review and payouts

Branch: feat/admin-winners

Prerequisites: B16, B18

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Build winner queue with filters, protected proof viewing, audit timeline, approval/rejection and mandatory rejection reason. Only approved unpaid winners may transition to paid. Require payout reference, actor and timestamp, with idempotency and concurrency protection. Treat mark-paid as recorded manual/demo settlement, not a bank transfer. Refresh subscriber views after review and preserve the evidence chain on resubmission. Prevent public access to uploaded evidence.

Files and integration

backend/src/modules/winners/; backend/src/modules/admin/;

frontend/src/pages/admin/WinnersPage.jsx; frontend/src/modules/admin/

API or routes: GET /api/admin/winners; POST /:id/approve, /reject, /mark-paid

Design references under UI Design: digital_heroes_admin_winner_verification_payout_operations; digital_heroes_admin_form_validation_governance_rules. Use each relevant screen.png and code.html where available.

Acceptance checks

Approve/reject/resubmit flow works; non-admin review fails; rejected or pending winners cannot be paid; repeated mark-paid cannot double-pay; queue and member views reflect the same state.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: add admin verification and payout controls

Commit body: describe the implemented behavior and actual verification.

PR title: feat: add admin verification and payout controls

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B22 Admin reporting

Branch: feat/admin-reports

Prerequisites: B19, B20, B21

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Implement total users, active subscriptions, current prize allocation, outstanding rollover, charity contribution totals and draw statistics with date filters and explicit metric definitions. Aggregate successful subscription allocation and independent donations separately, then reconcile charity totals without counting failed payments or annual revenue twice. Distinguish allocated prize pool, awarded winnings and paid payouts; do not add rollover twice. Build accessible charts/tables with empty/loading/error states and safe date-boundary handling.

Files and integration

backend/src/modules/admin/{admin.service,admin.controller,admin.routes}.js;

frontend/src/pages/admin/ReportsPage.jsx (addition);

frontend/src/pages/admin/AdminDashboardPage.jsx

API or routes: GET /api/admin/reports?from=&to=

Design references under UI Design: digital_heroes_admin_executive_reports_platform_analytics. Use each relevant screen.png and code.html where available.

Acceptance checks

Seeded fixtures reconcile with payment, contribution, draw and payout ledgers; date edges and no-data work; failed attempts are excluded; admin-only access is enforced.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: add reconciled admin reports and analytics

Commit body: describe the implemented behavior and actual verification.

PR title: feat: add reconciled admin reports and analytics

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B23 System errors and recovery

Branch: feat/error-states

Prerequisites: B17, B22

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Audit all routes and API calls for loading, empty, unauthorized, forbidden, not-found, inactive subscription, failed payment, missing score/draw and network errors. Add error boundary, retry guidance, consistent backend error codes and form focus management. Preserve unsaved input, clear stale member caches on session loss and avoid retrying writes without idempotency. Recovery-suite UI beyond login is optional unless backed by secure token-expiry, single-use and email delivery services; do not ship inert recovery links.

Files and integration

frontend/src/components/common/; frontend/src/pages/errors/ (addition); frontend/src/routes/; frontend/src/services/api.js; backend/src/middleware/

API or routes: 404, 401, 403, validation, payment, network and server errors

Design references under UI Design: digital_heroes_core_system_errors_foundation_patterns; digital_heroes_reusable_system_states_component_library; digital_heroes_subscriber_form_validation_state_matrix; digital_heroes_admin_form_validation_governance_rules. Use each relevant screen.png and code.html where available.

Acceptance checks

Inject 401/403/404/409/422/500 and offline conditions; each screen offers correct recovery without leaking traces or secrets; retry cannot duplicate payments/draws; keyboard focus remains usable.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: feat: complete system error and recovery states

Commit body: describe the implemented behavior and actual verification.

PR title: feat: complete system error and recovery states

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B24 Integration and end to end verification

Branch: test/integration

Prerequisites: B23 and all feature branches

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Add automated API, component and browser integration coverage against an isolated test database and isolated media/payment adapters. Cover register/charity/subscribe, five-score rolling window, simulation/publish, winnings/proof/review/payout and reports. Test authorization, concurrent score inserts, duplicate publish, stale preview, ledger conservation, annual allocations, cancelled/lapsed access and safe uploads. Run lint, tests and production build; document exact commands, failures, fixes and remaining external-service checks. Keep demo fixtures isolated from production.

Files and integration

backend/tests/ (addition); frontend/src/**/*test.jsx (addition); e2e/ (addition); CI configuration (addition)

API or routes: Full public, subscriber and administrator journeys

Design references under UI Design: All relevant UI Design screens; PRD evaluation checklist. Use each relevant screen.png and code.html where available.

Acceptance checks

All core journeys pass from clean seed; deterministic random/weighted fixtures pass; concurrent/idempotent cases pass; mobile/desktop smoke and keyboard checks pass; evidence records actual results.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: test: verify Digital Heroes end to end workflows

Commit body: describe the implemented behavior and actual verification.

PR title: test: verify Digital Heroes end to end workflows

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B25 Deployment configuration

Branch: chore/deployment

Prerequisites: B24

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Prepare Render backend, Vercel SPA routing, MongoDB Atlas and Cloudinary configuration for the root-plan demo. Configure exact CORS allowlist, HTTPS-aware cookie settings, CSRF protection, trusted proxy and health checks. Explain cross-site cookie limits and prefer a same-site custom domain or supported proxy arrangement. Document environment values by name without secrets, database indexes, safe seed procedure and rollback. Explicitly list unmet PRD Supabase/provider/new-account requirements. Do not claim deployment success without a real URL and smoke evidence; external deployment requires the user to supply or authorize the target.

Files and integration

backend deployment config; frontend/vercel.json (addition); .env.example files; docs/Deployment.md (addition); backend/src/config/env.js

API or routes: Frontend deep links; /api/health; cross-origin authenticated requests

Design references under UI Design: PRD deployment constraints and decisions D01 to D03. Use each relevant screen.png and code.html where available.

Acceptance checks

Production frontend build and deep links work; cookie login survives reload on intended domains; forbidden origins fail; health reflects database state; secrets stay server-side; production/demo mode is explicit.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: chore: configure deployment and production environment

Commit body: describe the implemented behavior and actual verification.

PR title: chore: configure deployment and production environment

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.

B26 Demo release

Branch: release/v1-demo

Prerequisites: B25

PR base: main (proposed; use actual default branch)

Implement complete runnable code for this branch in the existing workspace. Apply the Master coding prompt, inspect current files and dependencies, and preserve unrelated work. Use the specified structure; add missing files only with a clear purpose. Include real wiring, validation and error handling, with no TODO or fake-success path in the delivered scope.

Implement

Finish release documentation, verified setup/start/test commands, architecture/API summary, requirement coverage, demo guide and screenshots. Generate demo credentials through a controlled seed using local environment input; do not commit real passwords. Include actual hosted URLs only after verification, and disclose simulation mode and remaining PRD gaps. Run full user/admin smoke after deployment, inspect console/network errors, check responsive layouts and resolve release-blocking defects. Avoid new feature scope. Prepare release notes and rollback instructions.

Files and integration

README.md; docs/; demo seed; screenshots; bug fixes only

API or routes: Complete release smoke journey

Design references under UI Design: All shipped screens; PRD mandatory deliverables. Use each relevant screen.png and code.html where available.

Acceptance checks

Fresh setup is reproducible; signup/login/scores/dashboard/admin flows pass; demo account roles work; documentation matches implemented features; no secret files tracked; release gaps are explicit.

Explain the code and hand off

Explain what every new or changed function/component does, its inputs, outputs, side effects and caller; walk through the main data flow and important decisions in plain language. Add useful inline comments for non-obvious logic. Report files changed, exact run/test commands, actual results and remaining limitations.

Git commit subject: chore: prepare Digital Heroes v1 demo release

Commit body: describe the implemented behavior and actual verification.

PR title: chore: prepare Digital Heroes v1 demo release

Finish with a Git commit subject and body plus a PR title and PR description covering Summary, Changes, How to test, Results, Screenshots when relevant, and Risks. Prepare messages even if Git or remote access is unavailable. Do not claim a commit, push, PR or deployment happened unless executed; perform those actions only within the user-authorized workflow.